OmniAPO

System-wide audio processing with VST3 plugin hosting

Version 1.0.1
August 2026

Built from source. Offered as donationware — free to use, with no paid tier and no network access.

1. What this is

OmniAPO is a system-wide, kernel-level audio equalizer for Windows. It runs as an Audio Processing Object (APO) inside the Windows audio pipeline, so it processes every application's sound on its way to your speakers or headphones — there is nothing to install per app, and nothing an application can do to bypass it.

This edition is a private, from-source build of the **equalizerAPO64** fork (a double-precision, AVX2-optimized version of Equalizer APO 1.4.2). It keeps everything stock Equalizer APO does, and adds VST3 plugin hosting, five targeted bug fixes, an automatic self-repair for a known Windows Update regression, and a hardened real-time audio path. See “What's new” below for the details.

Because it sits in the pipeline itself rather than as a plugin inside any one application, the same filter chain applies uniformly everywhere — a browser tab, a game, a media player, or a VoIP call all pass through the same EQ.

2. What's new in this edition

VST3 plugin hosting

In addition to the VST2 hosting it inherited, this build hosts VST3 effect plugins directly inside the audio pipeline, alongside its own filters. See “Adding VST3 plugins” below.

Five bug fixes

- Renaming an audio device no longer breaks the device-specific config that was matched to it.
- The microphone no longer goes silent when effects are disabled for a capture device.
- A false “not installed” warning that could appear on launch is fixed.
- Registry backup files created by the installer are now valid and no longer collide with each other.
- Device names containing control characters no longer break installation.

APO registration self-repair

A Windows Update can silently wipe OmniAPO's registration from a device, which leaves audio unprocessed with no obvious cause. This build detects that condition and offers a one-click, consent-based repair instead of leaving it to be discovered by accident. Detection itself runs unelevated; only the repair step asks for your consent and administrator access.

A real-time audio path with no heap allocation

The audio thread that processes live sound no longer allocates memory while playing, removing a class of potential glitches and instability under load.

3. Features

- **Double-precision (64-bit) internal processing** — preserves accuracy when many filters stack, such as convolution, complex parametric EQ, or a full GraphicEQ.
- **AVX2-optimized DSP engine** — measured at roughly 54× real-time throughput on a modern desktop CPU, using under 2% of one core for a typical Peace + HeSuVi filter chain.
- **VST2 and VST3 plugin hosting**, in-process, alongside its own filters.
- **Parametric and graphic EQ**, plus convolution (impulse-response) filtering.
- **Per-audio-device configuration** — a separate filter chain for each playback or recording device.
- **Fully compatible with the Peace GUI and HeSuVi** presets.

4. How to use it

Getting started

OmniAPO installs two tools you use directly: the **Configuration Editor**, for editing and previewing filter chains, and the **Device Selector**, for choosing which playback and recording devices Equalizer APO processes.

Where configs live

Config files live in `C:\Program Files\EqualizerAPO\config`. OmniAPO reads `config.txt`, plus any files it includes, and reloads automatically whenever a file changes — no restart needed.

The basic idea

A config file is a plain-text list of directives, one per line, applied top to bottom. For example:

```
Preamp: -6 dB
Include: example.txt
GraphicEQ: 25 0; 40 0; 63 0; 100 0; 160 0; 250 0; 400 0; 630 0; 1000 0
```

`Preamp:` lowers the overall level before other filters are applied, giving them headroom to work with. `GraphicEQ:` sets gain at the listed frequency points, in Hz and dB pairs.

How Include: works

`Include:` pulls another file's directives in, verbatim, at that exact point in the chain. That lets you split a chain across several files (one per plugin or tool) and assemble the final order with a short list of `Include:` lines in `config.txt`.

5. Adding VST3 plugins

VST3 plugins live in the standard system-wide Windows location, `C:\Program Files\Common Files\VST3\` — visible to any VST3 host on the machine, not just OmniAPO.

Config syntax

```
VST3Plugin: Library <path> [CID <guid> | ClassIndex <n>]
```

`<path>` is the .vst3 file or bundle folder. Quote it if it contains spaces — the standard VST3 folder does (“Program Files”, “Common Files”):

```
VST3Plugin: Library "C:\Program Files\Common Files\VST3\LoudMax.vst3"
```

A path can also be given relative to OmniAPO's own VST plugin folder (the same one VST2 uses) instead of a full absolute path — useful if you keep plugins there rather than in the shared system-wide VST3 folder.

> **Note — CID vs. ClassIndex**
>
> Add `CID <guid>` or `ClassIndex <n>` when a plugin module exposes more than one effect class. `CID` is preferred for multi-effect modules: it keeps working even if the vendor reorders classes in a later plugin version, where a `ClassIndex` position could silently point at the wrong effect.

The easiest route

Add the plugin from the Configuration Editor instead of hand-writing the line: it enumerates the module's available classes and writes the correct `CID` for you, so you never need to look up a GUID by hand.

Already installed on this machine

LoudMax (a transparent limiter) and **Airwindows Consolidated** (a large free effects collection, exposed as one VST3 class with many algorithms selectable via a parameter) are already installed system-wide, ready to be added to a config.

6. Peace / HeSuVi notes

Peace manages its own file, `peace.txt`, and its own `Include:` line in `config.txt` — don't hand-edit either one.

> **Note — keep your additions separate**
>
> Put any of your own additions (VST3 lines, extra filters) in a separate file, and reference it with one `Include:` line of your own in `config.txt`. That way Peace can update its own section freely without ever touching or overwriting yours.

7. Troubleshooting

Most day-to-day problems fall into one of these four cases.

No sound, or an effect doesn't seem to apply

Open Device Selector and confirm OmniAPO is installed to the device you're actually listening on. Playback and recording endpoints are configured independently, so it's easy to have effects enabled on one but not the other.

After a Windows update, the EQ silently stops

Windows updates can wipe OmniAPO's registration from a device. If the self-repair prompt appears, approve it for a one-click fix. If audio is unprocessed but you didn't see a prompt, re-run Device Selector to re-register manually.

A plugin crashes audio

VST2 and VST3 plugins run in-process, inside the audio pipeline — so a broken or unstable plugin can take down the whole audio service, not just its own effect. Remove that plugin's line from the config file and save; audio recovers on the next config reload.

Logs

OmniAPO writes a log file to `%TEMP%\EqualizerAPO.log` (your user temp folder).

8. Reference

Full technical documentation

The complete engineering record for this build lives in the workspace repository's `docs\` folder:

- `final-engineering-audit.md` — the full engineering handover: change inventory, validation, deferred work.
- `vst3-verification.md` — VST3 test results against real plugins.
- `vst3-daily-use.md` — installing and activating VST3 plugins.

The same folder's `specs\` and `plans\` subfolders hold the original project spec and implementation plan this build was developed against.

Rebuilding from source

```
pwsh -File build.ps1 -Stage all
```

Stages: native, qt, package, installer. Requires VS2022 BuildTools with the VC.ATL component, Qt 6.10.1, and NSIS with its extensions — see the workspace repository's README for exact versions and setup notes.

Distribution

OmniAPO is offered as donationware and has no update channel of its own. It never contacts a server - there is no version check, no ping and no telemetry, and the binaries link no networking library at all. New versions are downloaded and installed by hand.

Each installed build is hash-verified against the binaries produced by that source tree, so what's running always matches what was built. There is no separate release process beyond that: the source tree on this machine *is* the release.